

Assignment 2 - Factory Method and Abstract Factory

Course: ShP-2216 Software Design Patterns

Institution: Astana IT University, School of Software Engineering

Academic year: 2026-2027 | Programme 6B06102 Software Engineering, Year 2, Trimester 4

Student: <Full name>

Group: <Group>

GitHub repository: <repository link>

Submitted commit: <commit hash>

1. Introduction

The application is a console logistics program. In a single run it does two things: it plans a delivery using either a truck or a ship, and it renders a button and a checkbox belonging to either the Windows or the macOS UI family. The delivery mode and the UI platform are chosen independently at startup, so all four combinations run without any code change.

Why Factory Method fits the transport part. There is a single product hierarchy - a `Transport` - and one varying decision: which concrete transport a given logistics branch uses. The steps around that decision (obtain a transport, hand it the cargo and destination, report the result) are identical for road and sea. Factory Method is designed exactly for this shape: the shared steps live in the creator's `planDelivery(...)`, and the single varying step is deferred to subclasses through `createTransport()`. A Simple Factory would place that decision in one conditional, which would have to be edited every time a transport is added; with Factory Method a new transport is a new subclass instead.

Why Abstract Factory fits the UI part. Here there is not one product hierarchy but two - `Button` and `Checkbox` - and the products must be *consistent*: a Windows button must never appear next to a macOS checkbox. Abstract Factory expresses precisely that constraint. Because one `GUIFactory` implementation supplies both creation methods, choosing `WindowsFactory` makes a mixed pair structurally impossible; the client cannot produce one even by mistake, because it never calls a component constructor at all.

The two patterns are complementary and not interchangeable: the transport side varies along one axis (one product, several implementations), while the UI side varies along two (several products that must belong to the same family).

2. UML class diagrams

Insert the rendered images here. Sources: `docs/uml-factory-method.puml` and `docs/uml-abstract-factory.puml`; a readable ASCII version of both is in `docs/uml-diagrams.md`.

2.1 Factory Method (Part A)

Roles shown in the diagram:

Pattern role	Class in the code
Product	<code>Transport</code> (interface)
Concrete products	<code>Truck</code> , <code>Ship</code>
Creator	<code>Logistics</code> (abstract)
Concrete creators	<code>RoadLogistics</code> , <code>SeaLogistics</code>
Client	<code>DeliveryApplication</code>

Relationships: `Truck` and `Ship` implement `Transport`; `RoadLogistics` and `SeaLogistics` extend `Logistics`; `Logistics` depends on `Transport` (it creates and uses one); each concrete creator has a `<<creates>>` dependency on its own transport; `DeliveryApplication` depends on the abstract `Logistics` type only.

Runtime path, matching the code exactly:

```

DeliveryApplication.run(cargo, destination)
-> Logistics.planDelivery(cargo, destination)    shared workflow, no concrete type named
    -> createTransport()                        overridden by RoadLogistics / SeaLogistics
        -> new Truck() | new Ship()
    -> Transport.deliver(cargo, destination)    invoked through the contract

```

2.2 Abstract Factory (Part B)

Pattern role	Class in the code
Abstract products	<code>Button</code> , <code>Checkbox</code> (interfaces, each with <code>paint()</code>)
Concrete products	<code>WindowsButton</code> , <code>WindowsCheckbox</code> , <code>MacOSButton</code> , <code>MacOSCheckbox</code>
Abstract factory	<code>GUIFactory</code> with <code>createButton()</code> and <code>createCheckbox()</code>
Concrete factories	<code>WindowsFactory</code> , <code>MacOSFactory</code>
Client	<code>DeliveryApplication</code> (constructor injection of <code>GUIFactory</code>)

`createButton()` is declared as returning `Button` and `createCheckbox()` as returning `Checkbox`, so the client is never handed a concrete type. `WindowsFactory` creates the Windows pair, `MacOSFactory` the macOS pair; the family grouping is visible in the diagram as the two `<<creates>>` dependencies leaving each factory.

3. Clean Code evidence

Five Clean Code practices are demonstrated below, each with an excerpt from the submitted code and the specific benefit it provides. Practices 4 and 5 are the two explanations connected to *Clean Code*, Chapter 6 (Objects and Data Structures). Summary:

#	Practice	Where it is shown
1	Meaningful names	<code>Logistics</code> , <code>RoadLogistics</code> , <code>Transport</code> , <code>GUIFactory</code>
2	Small methods, one responsibility each	<code>Main.main</code> <code>readChoice</code> <code>start</code>
3	No duplicated logic	<code>Logistics.planDelivery(...)</code> written once
4	Data abstraction (Chapter 6)	<code>DeliveryApplication</code> fields typed as interfaces
5	Objects and encapsulation (Chapter 6)	<code>Truck.deliver(...)</code> , private <code>ROAD_ROUTE</code>

1. Meaningful names

Names identify both the domain and the pattern role, so a reader can tell what a class is in the pattern just from its name.

```

// src/main/java/aitu/sdp/assignment2/factory/logistics/Logistics.java
public abstract class Logistics {
    protected abstract Transport createTransport();
    public String planDelivery(String cargo, String destination) { ... }
}

// src/main/java/aitu/sdp/assignment2/factory/logistics/RoadLogistics.java
public final class RoadLogistics extends Logistics { ... }

```

Benefit. `Logistics` / `RoadLogistics` / `SeaLogistics` name the creator role, `Transport` / `Truck` / `Ship` name the product role, and `GUIFactory` / `WindowsFactory` name the factory role. The parameters `cargo` and `destination` say what the values mean rather than `String a`, `String b`. During the defense the class name alone answers "which pattern role is this?", and no comment is needed to explain the mapping.

2. Small methods with one responsibility

Input handling, selection, construction, rendering, and delivery are separate methods in separate classes instead of one long `main`.

```

// src/main/java/aitu/sdp/assignment2/app/Main.java

```

```

public static void main(String[] args) {
    try (Scanner scanner = new Scanner(System.in)) {
        String deliveryMode = readChoice(args, 0, "...", scanner); // input only
        String platform      = readChoice(args, 1, "...", scanner); // input only
        start(deliveryMode, platform);                               // wiring + output
    } catch (UnsupportedChoiceException invalidChoice) {
        System.out.println(invalidChoice.getMessage());           // error reporting only
        System.out.println("Stopping without running the delivery.");
        System.exit(EXIT_INVALID_INPUT);
    }
}

```

Benefit. Each method does one thing at one level of abstraction: `readChoice` only obtains a raw string, `LogisticsSelector.select` / `GUIFactorySelector.select` only validate and choose, `DeliveryApplication.run` only renders and delivers. Changing how input arrives (for example, reading from a file) touches `readChoice` alone; the pattern code is untouched.

3. Avoid duplicated logic

The delivery workflow exists once, in the base creator, instead of being copied into each concrete creator.

```

// src/main/java/aitu/sdp/assignment2/factory/logistics/Logistics.java
public String planDelivery(String cargo, String destination) {
    Transport transport = createTransport(); // varies by subclass
    return transport.deliver(cargo, destination); // identical for every subclass
}

```

Benefit. `RoadLogistics` and `SeaLogistics` contain only the line that actually differs - which transport to instantiate. The workflow steps (obtain a transport, hand it the cargo and destination) are written once, so a change to the workflow cannot fall out of sync between the road and sea branches. The same reasoning applies to the UI side: repeated component creation lives inside `WindowsFactory` / `MacOSFactory` rather than being repeated at every call site.

4. Data abstraction (*Clean Code*, Chapter 6)

Callers depend on abstract contracts and never see a concrete implementation type. Chapter 6 argues that a class should *expose abstract interfaces that let users manipulate the essence of the data without having to know its implementation*.

```

// src/main/java/aitu/sdp/assignment2/app/DeliveryApplication.java
public final class DeliveryApplication {

    private final GUIFactory guiFactory; // abstraction, not WindowsFactory
    private final Logistics logistics;   // abstraction, not RoadLogistics
    private final Button button;         // abstraction, not WindowsButton
    private final Checkbox checkbox;     // abstraction, not WindowsCheckbox

    public DeliveryApplication(GUIFactory guiFactory, Logistics logistics) {
        this.guiFactory = guiFactory;
        this.logistics = logistics;
        this.button = guiFactory.createButton(); // obtained from the factory
        this.checkbox = guiFactory.createCheckbox(); // never `new WindowsButton()`
    }

    public String[] run(String cargo, String destination) {
        return new String[] {
            "UI family: " + guiFactory.platformName(),
            button.paint(), // through the interface
            checkbox.paint(), // through the interface
            logistics.planDelivery(cargo, destination) // through the abstract creator
        };
    }
}

```

Benefit. Every field is declared as an interface or abstract class, so the client has no way to depend on `WindowsButton` or `Truck`. There is no `instanceof`, no cast, and no branching on a concrete type anywhere in the client: the implementation is genuinely hidden, not merely wrapped in getters. Adding a Linux UI family therefore

requires no change to this class at all.

5. Objects and encapsulation (Clean Code, Chapter 6)

Behaviour is expressed as a method; internal state stays private and is not exposed through accessors that no caller needs.

```
// src/main/java/aitu/sdp/assignment2/factory/transport/Truck.java
public final class Truck implements Transport {

    private static final String ROAD_ROUTE = "highway route";    // private, no getter

    @Override
    public String deliver(String cargo, String destination) {    // behaviour, not data
        return "Truck delivers " + cargo + " to " + destination + " by " + ROAD_ROUTE;
    }
}
```

Benefit. Chapter 6 contrasts an *object*, which hides its data and exposes behaviour, with a *data structure*, which exposes its data and has no meaningful behaviour. `Truck` is an object in that sense: the route is private with no `getRoadRoute()`, and the only thing a caller can ask is "deliver this cargo". Had the class instead exposed `getRoadRoute()`, callers would begin assembling delivery messages themselves, and the delivery rule would leak out of the class. The same holds for `DeliveryApplication`, whose `button` and `checkbox` fields are private and are used only inside `run(...)`.

Note on stateless classes: `WindowsFactory`, `MacOSFactory`, `RoadLogistics`, and `SeaLogistics` hold no state at all. That is intentional - they exist purely to make a creation decision - and the rubric explicitly accepts stateless classes.

4. Verification evidence

All checks were run on one machine with the commands from the README. The full console transcript is in `docs/verification-transcript.txt`.

#	Input	Expected result	Actual result	Status
1	ROAD WINDOWS	Truck delivery; Windows button and checkbox	UI family: WINDOWS Rendering Windows button Rendering Windows checkbox Truck delivers laboratory equipment to Aktau warehouse by highway route	PASS
2	SEA WINDOWS	Ship delivery; Windows button and checkbox	UI family: WINDOWS Rendering Windows button Rendering Windows checkbox Ship delivers laboratory equipment to Aktau warehouse in containers by sea route	PASS
3	ROAD MACOS	Truck delivery; macOS button and checkbox	UI family: MACOS Rendering macOS button Rendering macOS checkbox Truck delivers laboratory equipment to Aktau warehouse by highway route	PASS
4	SEA MACOS	Ship delivery; macOS button and checkbox	UI family: MACOS Rendering macOS button Rendering macOS checkbox Ship delivers laboratory equipment to Aktau warehouse in containers by sea route	PASS
5	AIR WINDOWS	Clear validation message; no delivery	Unsupported delivery mode: 'AIR'. Supported values: ROAD, SEA. Stopping without running the delivery. Exit code 1; no UI rendered, no delivery performed	PASS

#	Input	Expected result	Actual result	Status
6	ROAD LINUX	Clear validation message; no UI construction	Unsupported UI platform: 'LINUX'. Supported values: WINDOWS, MACOS. Stopping without running the delivery. Exit code 1; no components created	PASS

Missing-input behaviour. Documented in the README: when a value is not supplied as an argument, the program prompts for it; if no console input is available, or the value is blank, it prints `Missing delivery mode. Supported values: ROAD, SEA.` (or the corresponding platform message) and stops with exit code 1. Verified: running `java -cp out aitu.sdp.assignment2.app.Main` with no arguments and no input printed `Missing input: no value was provided.` followed by `Stopping without running the delivery.` No default was substituted and no factory was left null.

Cross-platform note. Check 3 and Check 4 were executed on a single machine. Selecting the macOS family does not require a computer running macOS, because the components only print their platform and component type.

Additional observation: input is accepted case-insensitively (`road` / `macos` work), which is documented in the README as supported behaviour.

5. Conclusion

5.1 Comparing the two patterns

Factory Method and Abstract Factory both remove `new` from the client, but they answer different questions.

Factory Method concerns **one product hierarchy**. A single creation step inside an otherwise fixed algorithm is deferred to a subclass. The variation is expressed through inheritance: each subclass overrides one method. In this project that is the transport - `RoadLogistics` and `SeaLogistics` differ in exactly one line, and everything else is shared in `planDelivery(...)`.

Abstract Factory concerns **several related product families**. It groups multiple creation methods behind one interface so that the products a client receives are guaranteed to be consistent with one another. The variation is expressed through composition: the client is handed an object. In this project that is the UI - `WindowsFactory` supplies both a Windows button and a Windows checkbox, and family consistency is the whole point.

The relationship to *Simple Factory* is also worth stating: a Simple Factory concentrates the decision in one conditional inside a single `create(type)` method. Adding a product means editing that conditional, so the creation code is not closed to modification. Factory Method replaces the conditional with subclass overrides, so adding a product means adding a class. The small `switch` in `LogisticsSelector` and `GUIFactorySelector` is *not* a Simple Factory in this sense: it runs once at startup to translate a user's text choice into an object, and after that no branching on type occurs anywhere. Some mapping from external input to an object is unavoidable in any program that takes user input; what matters is that the delivery and rendering logic below it contains no conditionals, casts, or `instanceof` checks on product types.

5.2 Design reflection - what would change for three extensions

(a) Adding one more transport, e.g. `Drone` for air delivery.

- Add:** `Drone` implements `Transport` (one new class) and `AirLogistics` extends `Logistics` overriding `createTransport()` to return a `Drone` (one new class).
- Change:** one line in `LogisticsSelector.select(...)` to map `"AIR"` to `AirLogistics`, and the `SUPPORTED_MODES` constant so the validation message stays accurate.
- Unchanged:** `Logistics.planDelivery(...)`, `Transport`, `Truck`, `Ship`, the entire Abstract Factory side, and `DeliveryApplication` - which depends only on the abstract `Logistics` type and therefore cannot notice a new subclass. This is the payoff of Factory Method: the shared workflow is closed to modification.

(b) Adding one more UI family, e.g. `Linux`.

- *Add:* `LinuxButton` implements `Button`, `LinuxCheckbox` implements `Checkbox`, and `LinuxFactory` implements `GUIFactory` returning that pair (three new classes).
- *Change:* one line in `GUIFactorySelector.select(...)` plus the `SUPPORTED_PLATFORMS` constant.
- *Unchanged:* `Button`, `Checkbox`, `GUIFactory`, the existing `Windows` and `macOS` classes, and `DeliveryApplication`, which receives a `GUIFactory` and calls `createButton()` / `createCheckbox()` without knowing which family answered. Adding a *family* is the cheap direction for Abstract Factory.

(c) Adding one more UI product type, e.g. a text field.

This is the expensive direction, and it is a known trade-off of Abstract Factory rather than a flaw in this implementation.

- *Add:* a `TextField` interface with `paint()`, plus one implementation per family (`WindowsTextField`, `MacOSTextField`).
- *Change:* `GUIFactory` must declare `createTextField(): TextField` - and because that is a new method on the interface, **every** existing concrete factory must implement it. With two families that is two edits; with ten families it would be ten. `DeliveryApplication` must also change if the new component is to be rendered, since it decides which components the screen contains.
- *Unchanged:* `Button`, `Checkbox`, all four existing component classes, the selectors (no new input value is introduced), and the whole Factory Method side.

In short: **new families are cheap, new product types are expensive.** Both extension directions leave the startup selection code as the only place that mentions concrete classes, and leave the delivery and rendering logic untouched.

5.3 References

- 1 Lecture 2: Factory Method and Abstract Factory (Logistics and GUI examples; Clean Code, Chapter 6 slides), ShP-2216 Software Design Patterns, Astana IT University, 2026-2027.
- 2 E. Freeman, E. Robson. *Head First Design Patterns*, 2nd ed., O'Reilly, Chapter 4 - The Factory Pattern.
- 3 R. C. Martin. *Clean Code: A Handbook of Agile Software Craftsmanship*, Prentice Hall, Chapter 6 - Objects and Data Structures.
- 4 E. Gamma, R. Helm, R. Johnson, J. Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley - Factory Method and Abstract Factory.
- 5 Software Design Patterns syllabus 2026-2027 and the official course calendar in Moodle.